AI Applications in Research

Core Concepts and the Use of Claude Code in Empirical Work

Prepared as a scheduled research digest

30 July 2026

Abstract

This document summarises the main concepts surrounding the use of artificial intelligence—particularly agentic coding tools such as *Claude Code*—in empirical and quantitative research. It draws on the ongoing "Claude Code" series in Scott Cunningham's *Scott's Mixtape* Substack, which documents, in near real time, what works and what fails when autonomous and semi-autonomous agents are used to produce social-scientific research. The central organising idea is an economic one: coding agents have sharply lowered the *marginal cost of producing* research while leaving the *cost of verifying* it largely unchanged, and this shift in relative prices reorganises the entire research workflow around verification.

Contents

1 Framing: why AI changes the economics of research **1**

2 The key idea: "prose is not a constraint" **1**

3 Workflow concepts for AI-assisted empirical research **2**
- 3.1 The git diff as the "unit of verification" . . . 2
- 3.2 Adversarial "grilling" vs. cooperative "interviewing" . . . 2
- 3.3 Skills as reusable elicitation routines . . . 3
- 3.4 The Johari Window as a blind-spot tool . . . 3
- 3.5 Dashboards, checklists, and the "craft" of causal inference . . . 3

4 A design principle: zero errors as a constraint, not a goal **4**

5 Practical takeaways **4**

6 Sources and further reading **4**

1 Framing: why AI changes the economics of research

The recurring thesis is that by mid-2026 coding agents (Claude Code, Codex, and similar) have **dramatically reduced the marginal cost of producing empirical research**. Estimation, data wrangling, figure and table generation, and even research design can be driven with comparatively little human effort.

The complication is that **the cost of verification has not fallen proportionally**. Verification—confirming that a number, figure, or design is correct and traceable—still requires substantial human skill, and if anything requires *more* skill to do well, because agents cannot reliably verify their own output without human steering. This asymmetry is the conceptual core:

> Plummeting marginal cost of production + roughly constant cost of verification ⇒ verification becomes the binding task in AI-assisted empirical research.

A related worry is **"verification debt."** As agents produce more, and faster, the human reviewer accumulates a backlog of changes to check and begins to skim approvals rather than scrutinise them—so raw output can outrun the human capacity to validate it.

2 The key idea: "prose is not a constraint"

Perhaps the sharpest concept is the slogan **"prose is not a constraint."** Researchers routinely try to govern agents with written rules: instructions embedded in `CLAUDE.md` files, warnings inside prompts (e.g. "never produce a figure that cannot be traced to a local file"), or reference documents the agent is told to read.

The observation—reinforced by large-scale experiments across thousands of semi-autonomous agents—is that agents treat such instructions as *optional*. Their willingness to exercise discretion appears to grow as the task gets larger, as the context window fills with work, or seemingly at random. Concretely observed failures include:

- hallucinating a sample size (e.g. inventing "494 counties") rather than deriving it from the analytical sample;
- half-overwriting files despite explicit instructions not to;
- "programming in the air"—producing figures and tables in a terminal session without ever writing the code into the reproducible pipeline.

The explanation offered: an LLM faces **no consequence** for violating instructions. It cannot be grounded, docked, or otherwise penalised; throwing more prose at it (even frustrated prose) changes nothing. Written rules are therefore "as much a rule as laws without enforcement."

The corollary: structure binds, prose doesn't

If prose does not constrain, **tools and structure can**. The worked example is the `did` package in R (Callaway and Sant'Anna's estimator for staggered adoption): it *refuses* to run on panels where every unit is eventually treated, because the estimator requires not-yet-treated or never-treated comparison units. By contrast `lm` and `fixest` will happily estimate two-way fixed-effects models on the same data. The package itself—not any written instruction—is what constrains the agent's choices. This points toward **hooks and hard technical fences** that make forbidden actions *impossible*, rather than merely discouraged.

3 Workflow concepts for AI-assisted empirical research

3.1 The git diff as the "unit of verification"

A practice borrowed from Paul Goldsmith-Pinkham (household finance, NBER): treat the **git diff** as the atomic unit that a human approves or rejects. Invented sample sizes surface as green added

lines; clobbered files show as red; and lost work is recoverable with a single `git checkout`. Version control converts silent agent errors into visible, reviewable events. A caution is attached: diff-approval alone is inadequate if the reviewer, under verification debt, merely skims the diffs.

3.2 Adversarial "grilling" vs. cooperative "interviewing"

Two complementary elicitation styles are contrasted:

	Interview (`/covariates`, `/target`)	**Grill** (Goldsmith-Pinkham)
Stance	Cooperative: collect & build	Adversarial: doubt & attack
Probes	Your *knowledge* (pull it out)	Your *reasoning* (try to break it)
It trusts	Your answers (builds on them)	Nothing (makes you justify)
Stops when	Questions answered	The plan can't be broken
Output	A filled-in specification	A plan that survived attack

Interviewing leans on the *Polanyi paradox* (via David Autor): humans hold latent knowledge they cannot easily articulate, and generative AI can extract it. For example, a researcher may "know" which covariates satisfy conditional parallel trends without being able to state them; a structured interview surfaces that tacit knowledge. **Grilling** instead stress-tests a plan until it either breaks or survives, exposing weak points before any code is written.

3.3 Skills as reusable elicitation routines

Custom Claude Code "skills" (slash-commands) encapsulate repeatable procedures, e.g.:

- `/covariates` — a five-question (plus follow-ups) interview that proposes up to six covariates, grounded in the Heckman–Ichimura–Todd (1997) idea that the covariates which matter for parallel trends are those that (a) predict trends in the untreated potential outcome $Y(0)$ and (b) are severely imbalanced between treatment and control groups;
- `/target` — specifies the exact aggregated estimand (typically an average causal effect) that the design and estimator will target;
- `/amnesia` — an interviewer that re-orients the researcher on a project after time away, countering day-to-day forgetting.

3.4 The Johari Window as a blind-spot tool

The Johari Window (Luft & Ingham, 1955)—a 2×2 grid of what is known/ unknown to self versus others—is repurposed to compare one's own workflow against an expert's (here, Goldsmith-Pinkham's NBER practices). It surfaces the *shared spine* (both parties independently learned that verification is now central and that structure binds while prose does not), the *blind spots* (practices the expert uses that the researcher does not), and areas for adoption.

3.5 Dashboards, checklists, and the "craft" of causal inference

One documented personal approach is a highly **visual, checklist-based dashboard** that treats causal inference as a craft with explicit *stages of production.* Completed figures and tables become clickable "cards" pinned to the checklist; Claude Code is embedded directly in the dashboard for immediate access to "the machinery." The stated rationale is that the older "analog" style of hand-coding *generated verification knowledge as a by-product*; when the agent does the coding, that knowledge must be reconstructed deliberately—through visuals, tactile repetition, and locked/unlocked stages—especially for researchers who rely on external structure to remember and re-engage with a project.

4 A design principle: zero errors as a constraint, not a goal

A guiding formulation is:

maximise research output subject to **zero-error constraint.**

The distinction is deliberate. If minimising errors were merely a *goal* (minimise errors subject to a technology constraint), one implicitly accepts some level of *"optimal errors."* Treating zero errors as a *constraint* instead denies that optimal errors exist and forces the workflow to be *designed around* the constraint. Given how fast agents now produce research—and how far beyond a researcher's own knowledge base they can reach—tolerating optimal errors is argued to create serious problems in aggregate.

5 Practical takeaways

1. Assume the agent will ignore prose instructions some of the time; do not rely on `CLAUDE.md` or prompt warnings as guarantees.
2. Prefer **hard constraints**—packages that error out, hooks that forbid actions, version control that surfaces changes—over written rules.
3. Make the **git diff the unit of verification**, but guard against skimming diffs under verification debt.
4. Use **structured elicitation**: adversarial grilling to harden plans, cooperative interviewing to extract tacit domain knowledge.
5. Encode repeatable procedures as **skills** (`/covariates`, `/target`, `/amnesia`) so quality practices are reused, not re-improvised.
6. Deliberately **rebuild verification knowledge** that hand-coding used to supply for free (dashboards, checklists, staged workflows).
7. Treat **zero errors as a design constraint**, letting it shape the workflow rather than as a target to be traded off against speed.
8. Recognise that each researcher's "production function" differs—the aim is one's own production-possibility frontier, not mimicry of another workflow.

6 Sources and further reading

- Scott Cunningham, *Scott's Mixtape* Substack, "Claude Code" series: https://causalinf.substack.com/s/claude-code
- Latest instalment (30 Jul 2026), "Prose is not constraints and other things I learned from Paul Goldsmith-Pinkham's NBER talk on AI in empirical research": https://causalinf.substack.com/p/prose-is-not-constraints-and-other
- D. B. Rubin, "For Objective Causal Inference, Design Trumps Analysis."
- G. Imbens & Y. Xu, reassessment of the LaLonde training-program data.
- J. J. Heckman, H. Ichimura & P. Todd (1997), *Review of Economic Studies* — matching / difference-in-differences with covariate adjustment.
- P. Sant'Anna, "Pedro's Checklist" (informal lecture), featured in *Causal Inference: the Remix*.
- D. Autor, on generative AI, the Polanyi paradox, and middle-class expertise.

This is an independent conceptual summary compiled from publicly circulated newsletter content for research-reference purposes. Wording and framing are paraphrased; see the linked originals for the authoritative text.